3. Control Flow: if, for, and switch

03. Control Flow: if, for, and switch

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `03-control-flow`

Learning objectives

- Write idiomatic if statements with optional init
- Use Go's single loop construct (`for`) in all forms
- Use switch without accidental fallthrough
- Prefer clear control flow over clever nesting

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Go has one loop (`for`) and no accidental switch fallthrough. Clear control flow keeps infrastructure code debuggable under pressure.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `if` may include an init statement scoped to the if/else chain.
- `for` covers while, infinite, and C-style loops; `range` iterates collections.
- `switch` cases break by default; `fallthrough` is explicit and rare.
- Switch on true replaces long if/else chains cleanly.
- Braces are required even for one-line bodies.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-control && cd /tmp/gw-control
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

func main() {
    for i := 1; i <= 5; i++ {
        switch {
        case i%15 == 0:
            fmt.Println("FizzBuzz")
        case i%3 == 0:
            fmt.Println("Fizz")
        case i%5 == 0:
            fmt.Println("Buzz")
        default:
            fmt.Println(i)
        }
    }
}
```

Run it:

```
go run .
```

Expected output:

```
1
2
Fizz
4
Buzz
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Rewrite a Python `while` as `for cond { }`.
2. Use `if n := len(xs); n == 0` and note that n is out of scope after.
3. Implement Classify with switch-on-true.
4. Write an infinite `for { }` with `break` after three iterations.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

if

```go
if n := len(items); n == 0 {
    return ErrEmpty
} else if n > 100 {
    return ErrTooMany
}
// n is not in scope here
```

No parentheses around conditions. Braces are required.

for — the only loop

```
for i := 0; i < 10; i++ { }
for cond { }           // like while
for { }                // infinite; break/return to leave
for i, v := range xs { }
```

switch

```
switch day {
case 1, 2, 3:
    // weekday start
case 6, 7:
    // weekend
default:
}
// cases do NOT fall through unless you write fallthrough
```

Switch on true is a clean if-else chain:

```
switch {
case score >= 90:
    grade = "A"
case score >= 80:
    grade = "B"
default:
    grade = "C"
}
```

Failure modes to watch while coding

- Expecting C-style fallthrough in switch
- Writing `while` — it does not exist; use `for cond`
- Forgetting braces (required even for one-liners)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- No `while` keyword — use `for cond`.
- Switch does not fall through like C; closer to Python 3.10 `match` without fallthrough.
- No ternary `a if c else b` — use if/else or a small function.
- `range` on a slice yields index, value — not just values unless you use _.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Prefer early `return` / `continue` over deep nesting. Flat control flow is an idiomatic Go readability feature.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Control Flow: if, for, and switch” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Expecting C-style fallthrough in switch
- Writing `while` — it does not exist; use `for cond`
- Forgetting braces (required even for one-liners)

Exercises

Work in `exercises/03-control-flow/`. Read that folder’s `README.md` before you code.

What you will build

Implement FizzBuzz(n) for values 1..n and Classify(n)→neg|zero|pos.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/03-control-flow/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Rewrite a while-loop as Go `for`
2. When is switch-on-true preferable to if/else?

3. What is the scope of a variable declared in `if init`?

Further reading

- https://go.dev/tour/flowcontrol